LECTURE 1

Wednesday, July 22, 2026 10:49 PM

Why this course

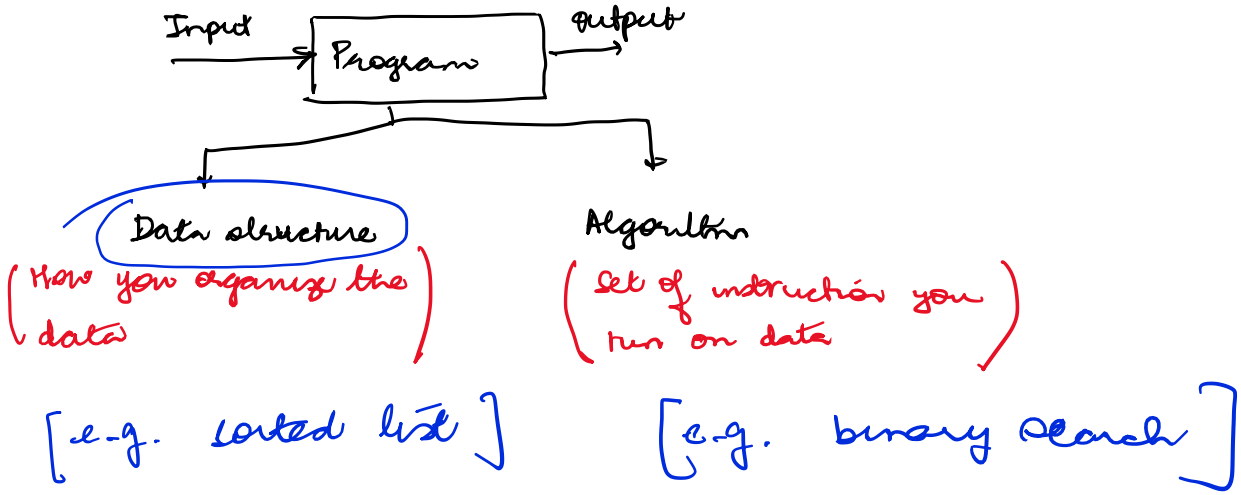
Prob 1: Suppose we want to check "is this name in my list of 1,000,000 names"

Prog #1 : Unsorted list, linear scan

worst case: 1M comparison

Prog #2 : Sorted list, binary search

worst case: 20 comparison



Other Problems

Problem 2: You are building the backend for X (Twitter). 50,000 new tweets arrive every single second. *How do you maintain a constantly updating, perfectly accurate list of the "Top 10 Trending Hashtags" at any given millisecond, without having to stop and sort a list of millions of words?*

Problem 3: You are programming an internet backbone router. Over 100 million data packets arrive every single second.
The Problem: For every single packet, how do you instantly match its destination IP address against a global directory of over 900,000 network routes, without scanning that directory from top to bottom 100 million times a second?

Problem 4: You are programming a massive open-world video game with 500,000 objects on the map (trees, buildings, enemies, vehicles). When a player throws a grenade, how does the game instantaneously figure out which 12 objects are within the 5-meter blast radius, without calculating the distance between the grenade and every single object on the map?

This course: write code that scales efficiently to millions of objects

② Which programming language?

Python

ANALOGY: live translator

Interpreted language

Code is read and executed line by line

Relatively slow

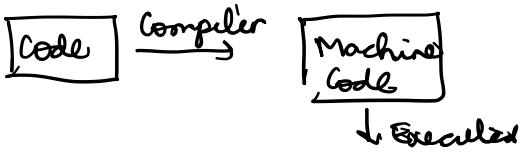
but high-level, more natural

* Compiler can optimize while translating

ANALOGY: translating a pre-written speech *

Compiler-based language

(C, Rust)



Code in Python

```
x = 5
x = "hello"
```

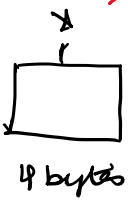
x → ~~5~~ "hello"

Dynamically typed language

<Name> → <Type, object>

code in C

```
int x = 5;
x = "hello"
```



Statically typed language

<Name, Type> → <object, type>

Prog

```
a = [1,2,3]
b = a
b.append(4)
print(a)
```

a → [1,2,3,4]
b →

a → b

[1,2,3,4,1,2,3,4]

```
b = a+b
print(a)
```

I - [1,2,3,4]

II - [1,2,3,4,1,2,3,4]

III - [2,4,6,8]

```
b += a
print(a)
```

a → [1,2,3,4]

b → [1,2,3,4,1,2,3,4]

Homework

↳ Namespace :

global Namespace

functions when executed

they get assigned their own namespace

```
def add_song(p, title):
   p.append(title)

playlist = ["Song A", "Song B"]
add_song(playlist, "Song C")
print(playlist)
```

print(p) Homework

Recap

→ Why this course
→ Python
→ Object assignment, Namespaces

Next class

↳ How to do sorting ?

↳ How to reason about runtime complexity of a program ?